Tecnológico de estudios Superiores de Villa Guerrero.

Ingeniería en Sistemas Computacionales

UNIDAD DE APRENDIZAJE:

SISTEMAS OPERATIVOS

PRÁCTICA:

PRACTICA 5. Rice Condition.C

PRESENTA:

GARCIA ALBARRAN ARACELI ADRIANA

GRUPO:

3501

DOCENTE:

ISC. REYNOSO ELIAS JUAN

LA FINCA, VILLA GUERRERO, ESTADO DE MÉXICO.

OCTUBRE DE 2026

INDICE

INDICE 2
INTRODUCCION 3
RICE CONDITION 4

INTRODUCCION

En el ámbito de la computación moderna, la programación concurrente permite ejecutar múltiples tareas de manera simultánea para optimizar el rendimiento del procesador. Sin embargo, cuando varios hilos de ejecución comparten un mismo espacio de memoria y necesitan modificar datos comunes, surgen desafíos críticos conocidos como condiciones de carrera. Estos problemas provocan que los resultados finales sean inconsistentes o erróneos, ya que las operaciones de lectura y escritura se superponen sin control. Para evitarlo, es indispensable implementar mecanismos de sincronización, siendo los mutex (exclusión mutua) una de las soluciones más utilizadas y eficientes en lenguajes como C mediante la biblioteca POSIX Threads (pthread.h).

RICE CONDITION

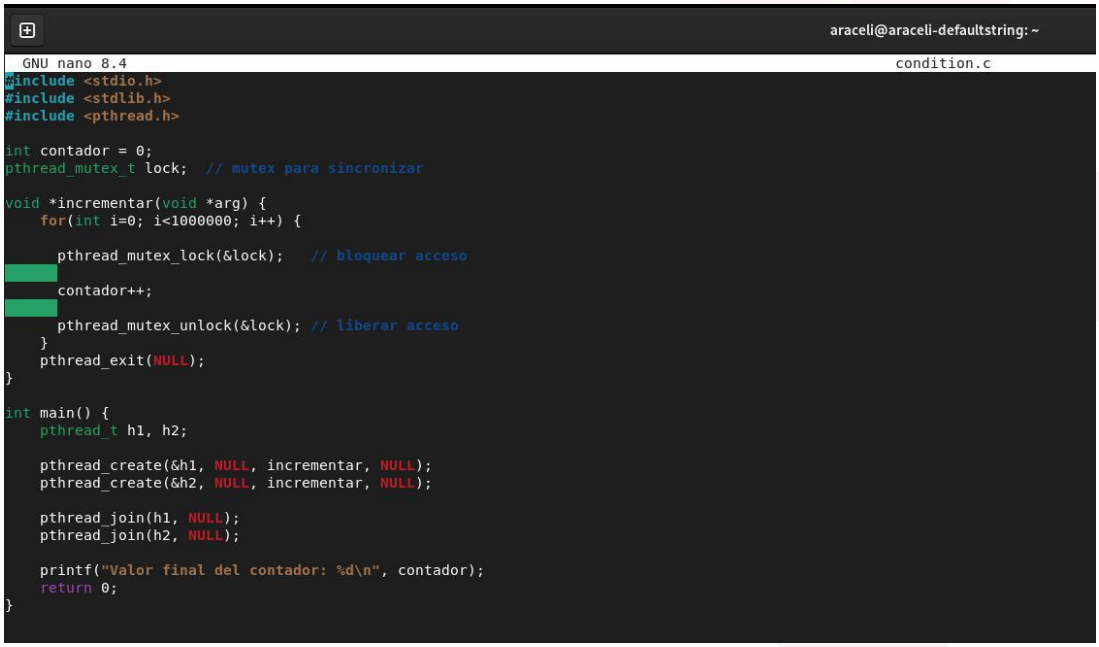
Este programa sirve para comprender cómo se utiliza un cerrojo mutex (pthread_mutex_t) para sincronizar hilos y evitar condiciones de carrera al acceder y modificar una variable global compartida.

Componentes principales:

Librerías y Variables Globales: El código incluye la biblioteca estándar de POSIX Threads (pthread.h). Se define una variable global entera llamada contador inicializada en 0 y una estructura pthread_mutex_t lock destinada a la sincronización.

Función del Hilo: Define la tarea que ejecutan los hilos concurrentes mediante un bucle de un millón de iteraciones (for). Utiliza pthread_mutex_lock(&lock) para bloquear el acceso a la sección crítica, incrementa de forma segura la variable con contador++, y libera el acceso mediante pthread_mutex_unlock(&lock), concluyendo con pthread_exit(NULL).

Función Principal (main): Se encarga de declarar los hilos h1 y h2 de tipo pthread_t, crearlos con pthread_create para ejecutar la función incrementar, sincronizar su finalización utilizando pthread_join, e imprimir en pantalla el valor final obtenido en el contador mediante printf.



```
araceli@araceli-defaultstring:~$ nano condition.c
araceli@araceli-defaultstring:~$ ./condition.exe
Valor final del contador: 2000000
araceli@araceli-defaultstring:~$
```